E²创智天地

小型智能交通装置的设计与制作基础

智能控制器 Arduino

常宝林

基于Arduino智能小车的视觉巡视、巡线行驶和避障

设计目标：

I. 使基于Arduino智能小车可以正常行驶并可实现视觉巡线；（40%）

II. 三路巡线模块按照设定线路行驶（30%）

III. 基于超声传感模块使得小车可以绕开障碍物正常行驶；（30%）

设备器件：

基于Arduino控制板智能小车；三路巡线模块；超声传感模块等

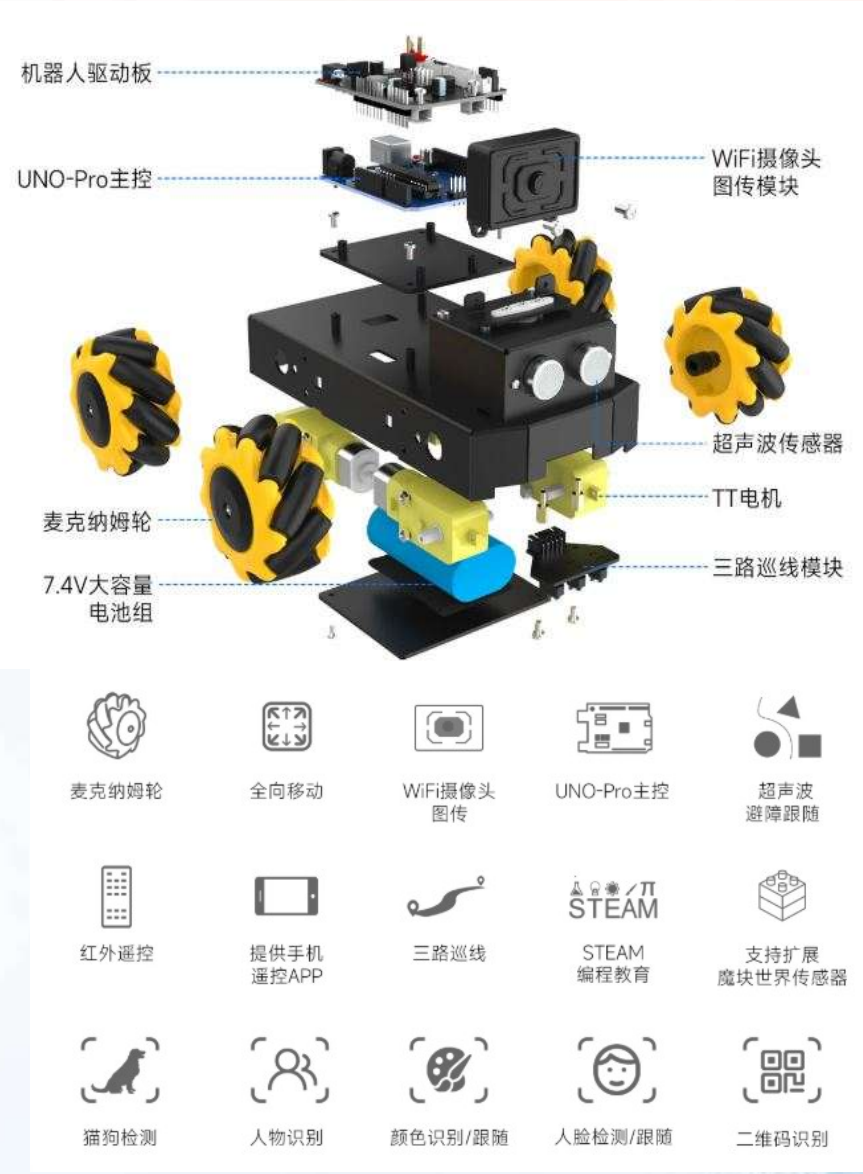
设计报告：

应包括但不限于：1、组员贡献互评；2、项目简介；3.项目成果；4.项目实施过程（附过程照片）、关键技术及实现效果；5、个人主要工作；6.收获与心得；7.意见与建议。

Arduino智能小车







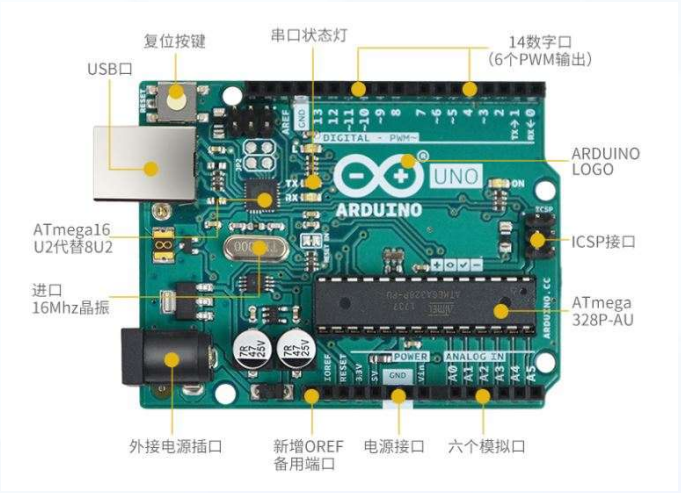
Arduino智能小车

主控管脑子，驱动管力气



驱动板：“手脚”，负责把主控的弱电小信号，放大/转换成足够功率的电信号，去驱动电机、屏幕、LED等大功率负载，并保护主控、隔离强弱电、提供保护与控制等。

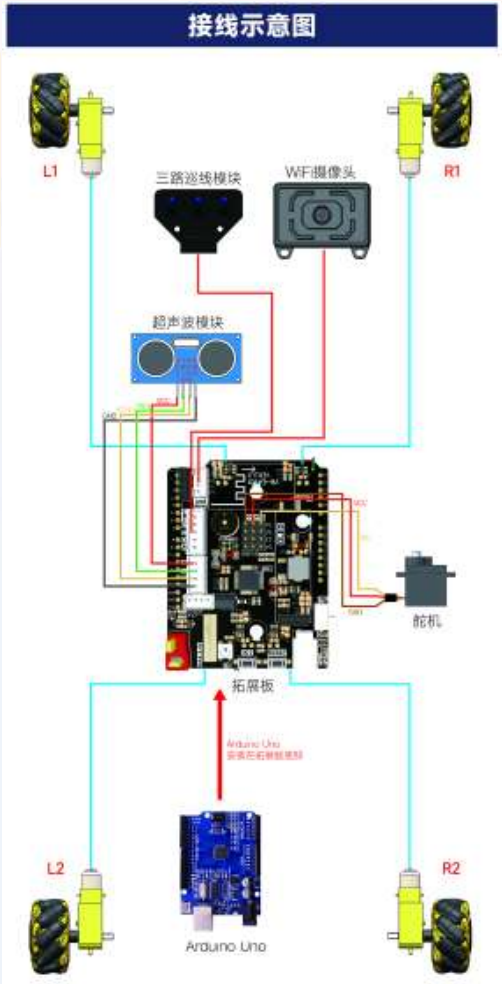
主控板：“大脑”，负责发号施令、逻辑控制、信号处理和人机交互等。



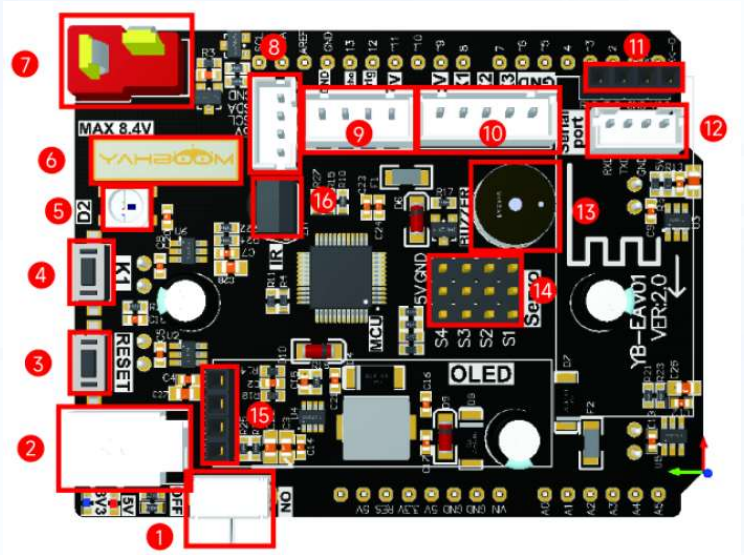
Arduino主控板

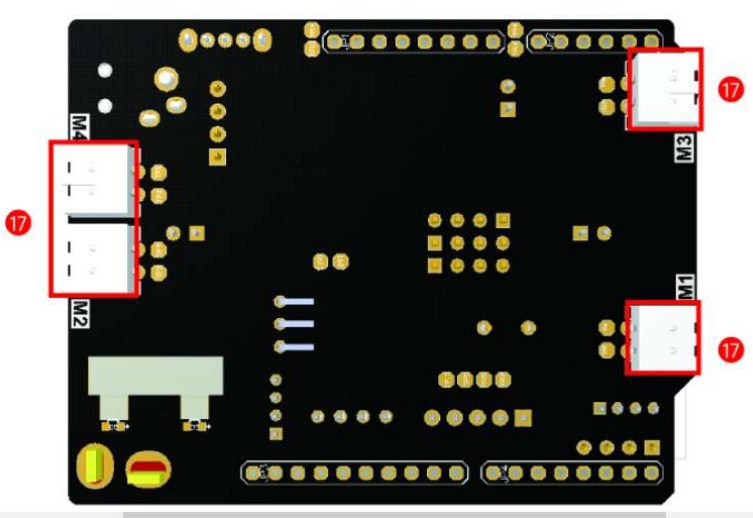


扩展驱动板



Arduino智能小车





①：拓展板电源开关 → 控制小车电源开关

②：电池充电接口 → 给小车电池组充电

③：RESET按键 → Arduino IDE复位

④：K1按键 → 可自定义按键功能

⑤：RGB灯 → 可控制显示不同颜色

⑥：LOGO标志灯 → 显示Yahboom标志

⑦：T型供电接口 → 给整个小车供电

⑧⑮：I2C接口 → 可外接I2C设备

⑨：超声波模块接口 → 外接超声波模块

⑩：三路巡线模块接口 → 外接三路巡线模块

⑪：串口接口 → 可外接串口通讯设备

⑫：串口接口→ 外接WiFi摄像头

⑬：无源蜂鸣器 → 可控制蜂鸣器发声

⑭：PWM舵机接口 → 外接PWM舵机

⑯：红外接收器 → 接收红外遥控器信号

⑰：电机接口 → 外接小车四个电机

搭建开发环境

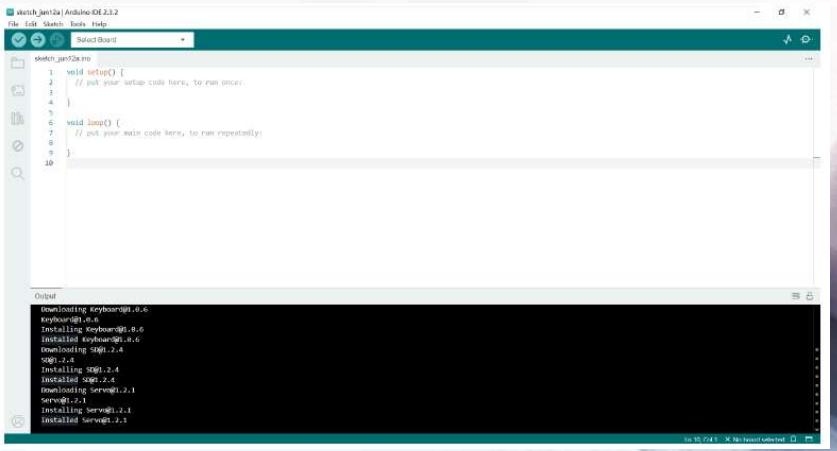
➢ 下载/安装*Arduino IDE*

(1)

名称	修改日期	类型	大小
Arduino-ide_2.3.2_Win_64.zip	2025/3/17 14:28	360压缩 ZIP 文件	145,819 KB
CH341SER.zip	2025/3/17 14:26	360压缩 ZIP 文件	597 KB
UartAssist 5.0.3.zip	2025/3/17 14:26	360压缩 ZIP 文件	609 KB

(2) **Arduino IDE官网： https://www.arduino.cc/en/software**



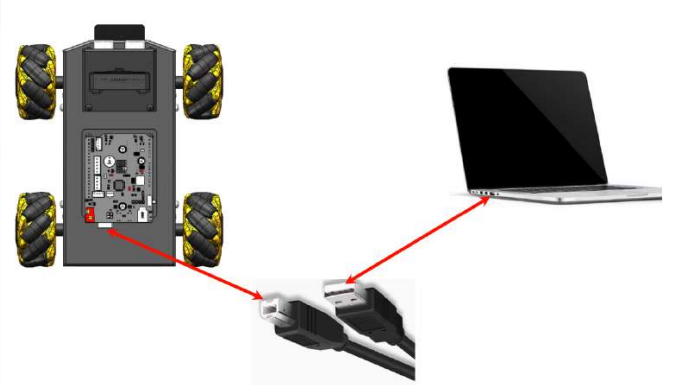


Arduino IDE界面

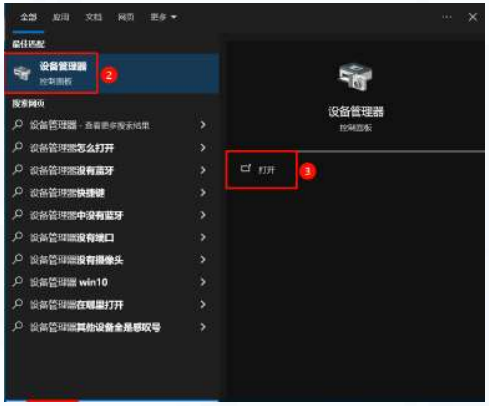
搭建开发环境

➢ 安装驱动

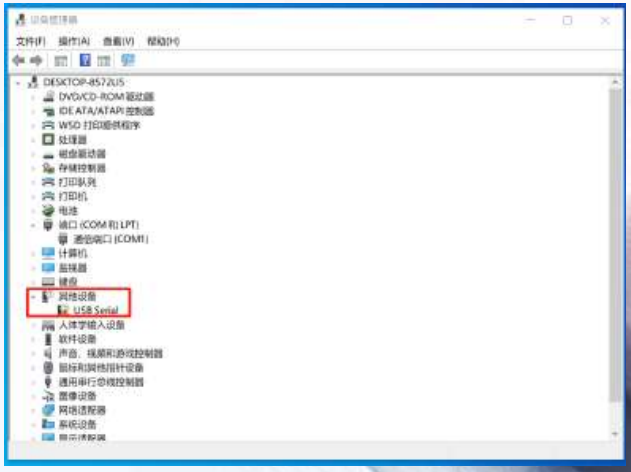
（1）建立物理连接



（2）打开设备管理器



（3）未安装CH340驱动，系统设备管理器会在新设备的左侧图标显示感叹号标志！



（4）解压并以管理员身份打开【软件资源：软件工具】中的**CH341SER.EXE**



（5）重新打开设备管理器，看是否可以正常识别CH340设备



记住这里的COM口的号，
例如：COM3

开发环境*Arduino IDE*

Arduino IDE——用于Arduino系列源代码开发，免安装

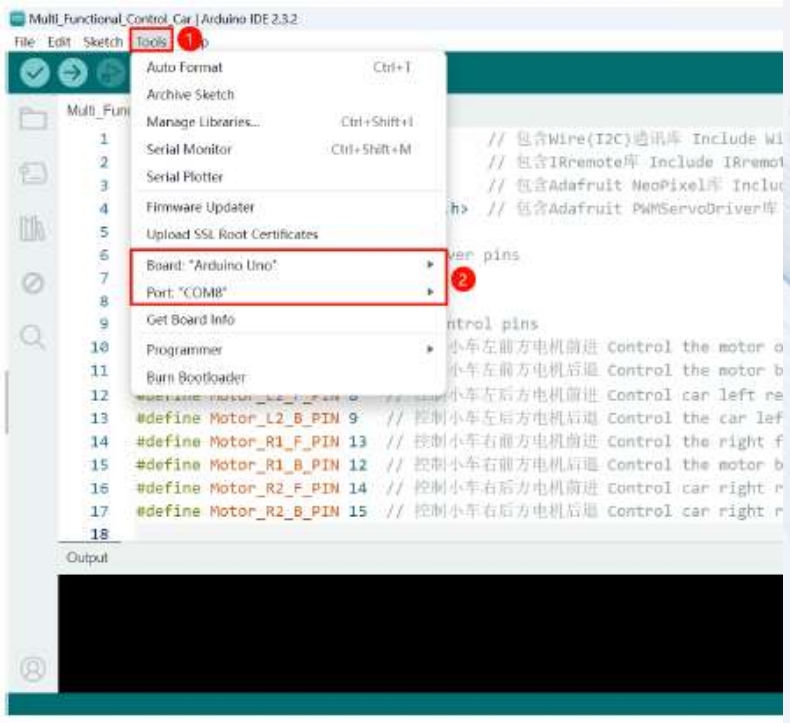


新建/打开工程

➢ **选择开发板对应的型号和串口号：**

I. **开发板选择：** *点击Tools → Board → Arduino AVR Boards → Arduino Uno*

II. **串口号选择：** *点击Tools → Port → COM8*

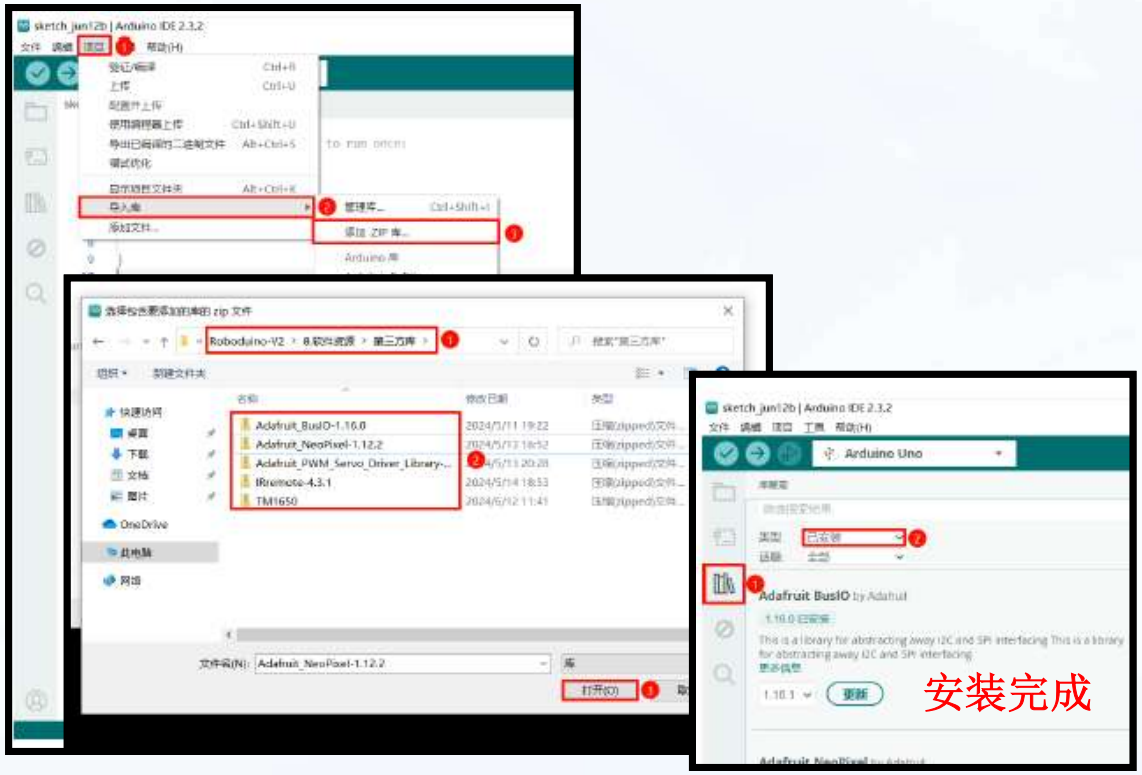


开发环境*Arduino IDE*

安装第三方库

整个小车案例需要用到Adafruit_NeoPixel 、Adafruit_PWM_Servo_Driver_Library 、TM1650 以及IRremote 等库。

（1）添加.zip库（本地有）



（2）软件管理库，在管理库搜索并点击安装。

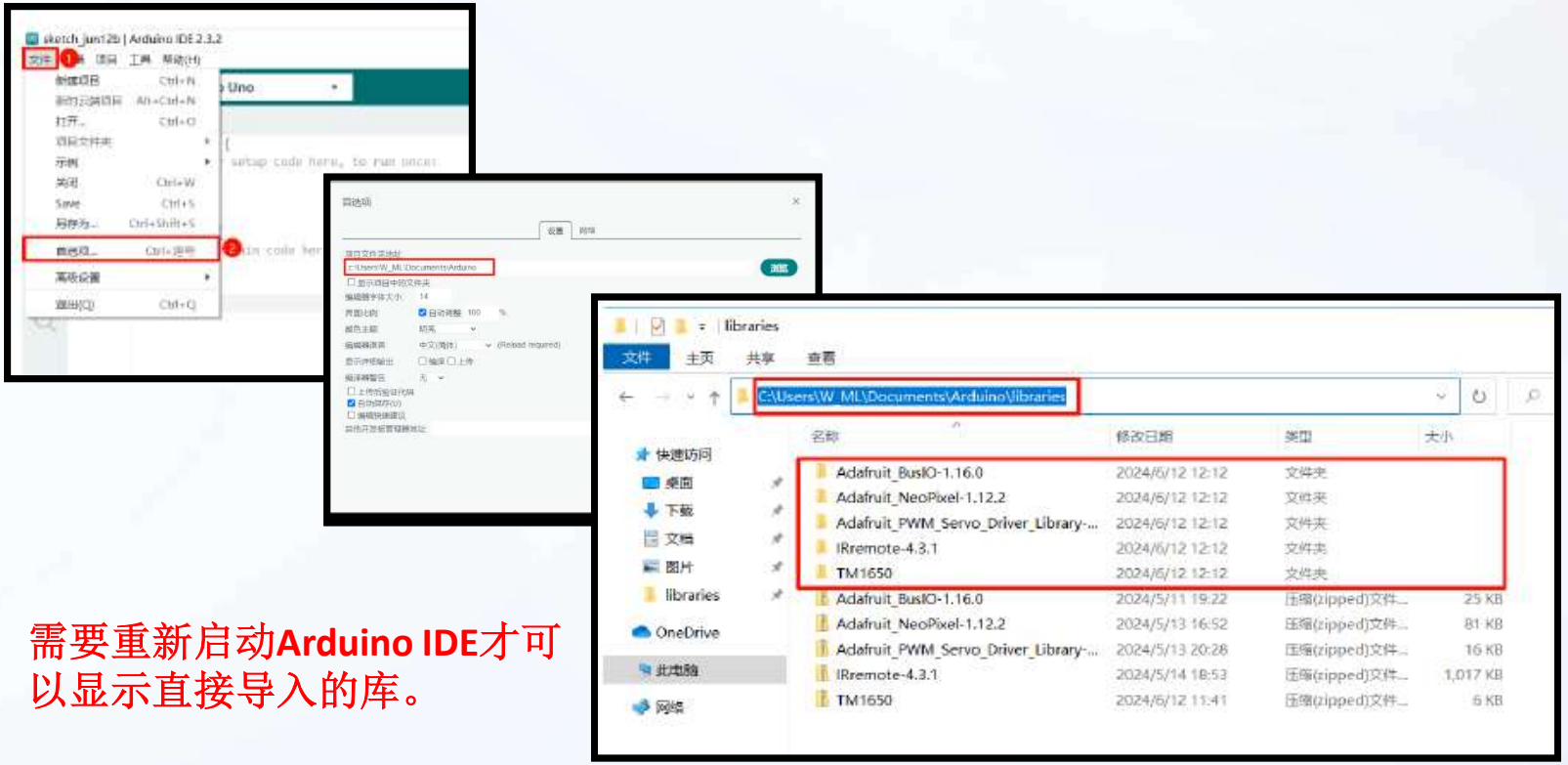




开发环境*Arduino IDE*

（3）直接导入库

——找到库所在文件夹并将软件库解压到**libraries** 目录下。



需要重新启动**Arduino IDE**才可以显示直接导入的库。

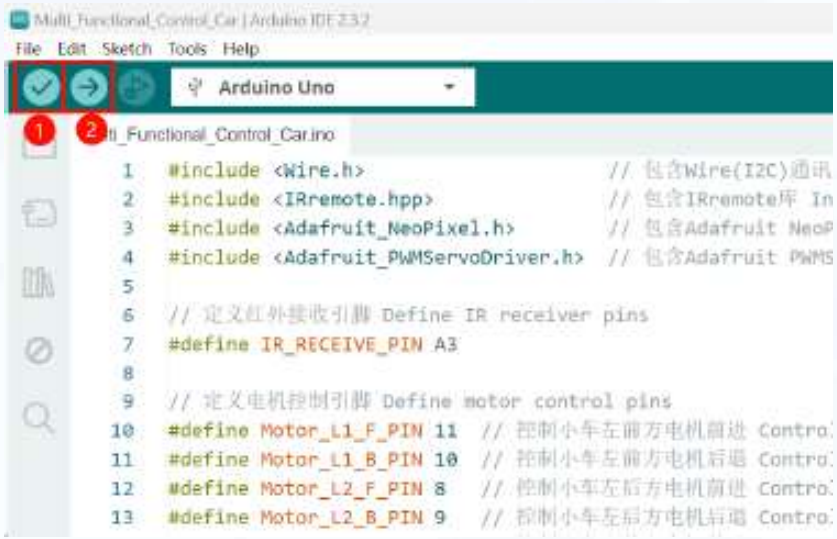
使用第三方库

--将库对应的头文件包含进去即可使用。

例如：

```
#include <TM1650.h>
#include <IRremote.hpp>
#include <Adafruit_NeoPixel.h>
#include <Adafruit_PWMServoDriver.h>
```

程序编译下载



程序编译、上传（下载）

I. 编译（*Verify*）选项：点击① → 编译成功，*Arduino IDE* 右下角会出现编译完成（*Done compiling*）弹窗！

II. 上传（*Upload*）选项：点击② → 上传成功，*Arduino IDE* 右下角会出现上传完成（*Done uploading*）弹窗！

程序上传到*Arduino Uno*会失败的原因？

I. 一般是由于串口被占用或者选择错误的开发板和串口号导致的；我们可以先检查*Arduino IDE*选择的开发板型号和串口号是否正确；

II. 检查*WiFi*摄像头是否安装在拓展板上占用串口；（程序上传时*WiFi*摄像头的串口连接线要拔掉）

程序设计与编写

*Arduino*编程的整体结构：

程序头部
- #include 库文件
- #define 常量
- 引脚定义、全局变量

初始化模块 setup ()
- 仅运行一次
- 硬件初始化、模式设置

主任务模块 loop ()
- 循环运行
- 业务逻辑、传感器采集、设备控制

自定义函数（可选）
- 写在 loop 外面
- 封装重复功能，被 setup/loop 调用

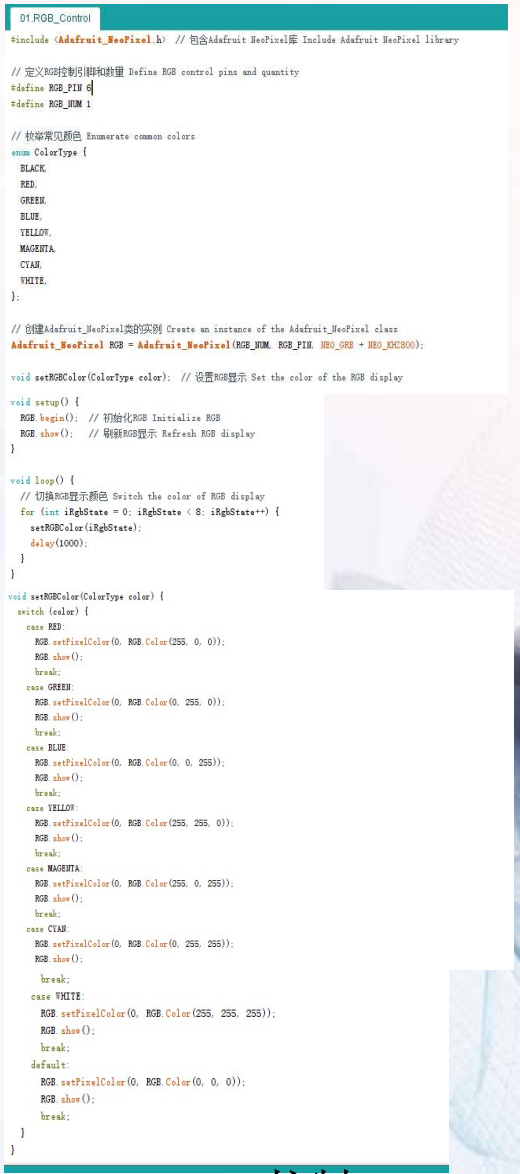


新建工程初始界面

sketch_may5a.ino

```
int led = 13; //LED 引脚定义
void setup() {
// put your setup code here, to run once:
pinMode(led, OUTPUT); //设置LED 为输出模式
digitalWrite(led, HIGH); //初始化设置LED 引脚为高电平
}
void loop() {
// put your main code here, to run repeatedly:
digitalWrite(led, HIGH); //初始化设置LED 引脚为高电平
delay(1000); //等待1000 毫秒
digitalWrite(led, LOW); //初始化设置LED 引脚为高电平
delay(2000); //等待2000 毫秒
}
```

板载LED闪烁示列



```
01.RGB_Control
#include <Adafruit_NeoPixel.h>  // 包含Adafruit NeoPixel库 Include Adafruit NeoPixel library

// 定义RGB控制引脚和数量 Define RGB control pins and quantity
#define RGB_PIN 6
#define RGB_NUM 1

// 枚举常见颜色 Enumerate common colors
enum ColorType {
  BLACK,
  RED,
  GREEN,
  BLUE,
  YELLOW,
  MAGENTA,
  CYAN,
  WHITE,
};

// 创建Adafruit_NeoPixel类的实例 Create an instance of the Adafruit_NeoPixel class
Adafruit_NeoPixel RGB = Adafruit_NeoPixel(RGB_NUM, RGB_PIN, NEO_GRB + NEO_KHZ800);

void setRGBColor(ColorType color);  // 设置RGB显示 Set the color of the RGB display

void setup() {
  RGB.begin();  // 初始化RGB Initialize RGB
  RGB.show();   // 刷新RGB显示 Refresh RGB display
}

void loop() {
  // 切换RGB显示颜色 Switch the color of RGB display
  for (int iRgbState = 0; iRgbState < 8; iRgbState++) {
    setRGBColor(iRgbState);
    delay(1000);
  }
}

void setRGBColor(ColorType color) {
  switch (color) {
    case RED:
      RGB.setPixelColor(0, RGB.Color(255, 0, 0));
      RGB.show();
      break;
    case GREEN:
      RGB.setPixelColor(0, RGB.Color(0, 255, 0));
      RGB.show();
      break;
    case BLUE:
      RGB.setPixelColor(0, RGB.Color(0, 0, 255));
      RGB.show();
      break;
    case YELLOW:
      RGB.setPixelColor(0, RGB.Color(255, 255, 0));
      RGB.show();
      break;
    case MAGENTA:
      RGB.setPixelColor(0, RGB.Color(255, 0, 255));
      RGB.show();
      break;
    case CYAN:
      RGB.setPixelColor(0, RGB.Color(0, 255, 255));
      RGB.show();
      break;
    case WHITE:
      RGB.setPixelColor(0, RGB.Color(255, 255, 255));
      RGB.show();
      break;
    default:
      RGB.setPixelColor(0, RGB.Color(0, 0, 0));
      RGB.show();
      break;
  }
}
```

RGB控制

程序设计与编写

以板载LED为例

I. 引脚定义，定义LED 为13 号引脚。int led = 13; //LED 引脚定义；

II. Setup 初始化，将LED 引脚定义为输出模式，并开机时为高电平状态。Setup 函数内容为主板启动后首先执行的代码，串行执行一次。

III. Loop 循环函数，设置LED 引脚为高，延续0.5S 后设置LED 引脚为低，再延迟1S。Loop 函数为执行完setup 初始化后，循环执行的代码。

代码内容如下：

```
int led = 13; //LED 引脚定义
void setup() {
// put your setup code here, to run once:
pinMode(led, OUTPUT); //设置LED 为输出模式
digitalWrite(led, HIGH); //初始化设置LED 引脚为高电平
}
void loop() {
// put your main code here, to run repeatedly:
digitalWrite(led, HIGH); //初始化设置LED 引脚为高电平
delay(1000); //等待1000 毫秒
digitalWrite(led, LOW); //初始化设置LED 引脚为高电平
delay(2000); //等待2000 毫秒
}
```

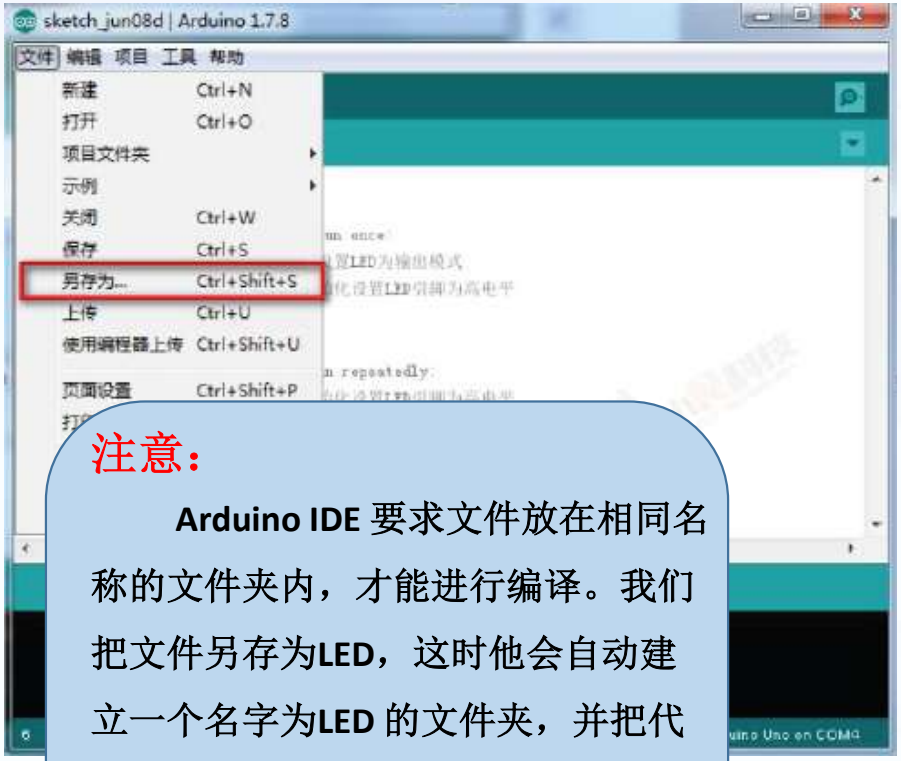
▲注意：

① 编写代码过程，需要使用英文输入法，特备是输出符号的时候。IDE 是不识别中文字符的。

② 注释的内容可以用中文，因为不参与编译过程。两个//为注释字符。

程序设计与编写

文件的保存



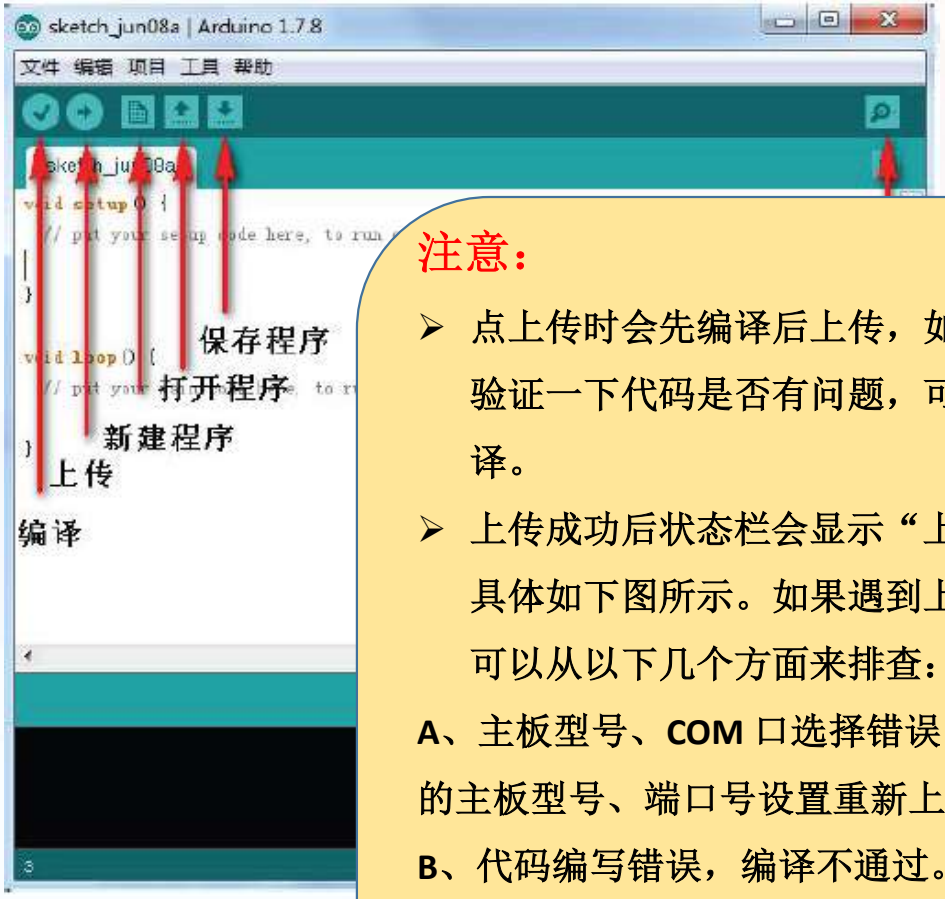
注意：

Arduino IDE 要求文件放在相同名称的文件夹内，才能进行编译。我们把文件另存为**LED**，这时他会自动建立一个名字为**LED** 的文件夹，并把代码文件保存在里面。

步骤为：文件→ 另存为→ 桌面→ **LED**

此时桌面就有了**LED** 文件夹

编译上传



注意：

- 点上传时会先编译后上传，如果只是想验证一下代码是否有问题，可以只点编译。
- 上传成功后状态栏会显示“上传成功”，具体如下图所示。如果遇到上传失败，可以从以下几个方面来排查：

A、主板型号、COM 口选择错误。按照正确的主板型号、端口号设置重新上传即可；

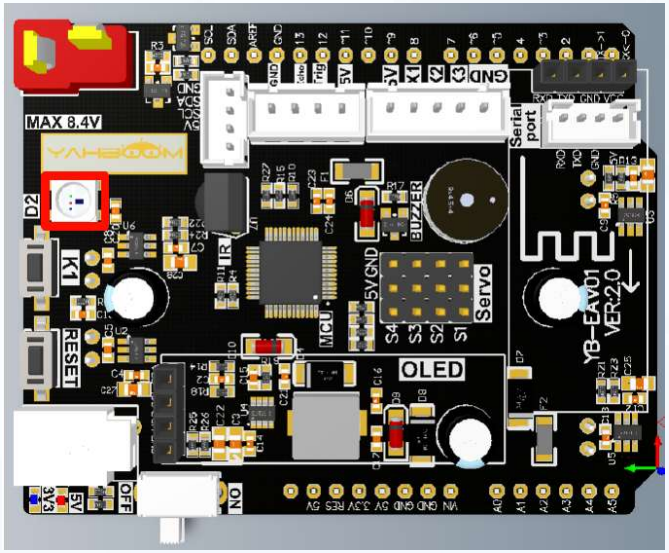
B、代码编写错误，编译不通过。根据错误提示修改后（一般是用了中文输入法输入了“；”）重新上传即可；

C、串口占用，造成数据异常，无法正常上传。拔掉wifi 数据线，重新上传即可。

程序设计与编写

7彩RGB控制

1. 包含Adafruit_NeoPixel 库
2. 定义RGB控制引脚和数量
3. 枚举常见颜色
4. 创建 Adafruit_NeoPixel 类的实例= 定义灯带，固定格式：
   Adafruit_NeoPixel pixels(灯珠数量, 控制引脚, 灯珠类型);
5. 引入封装好的设置颜色接口函数
6. 初始化
7. 循环运行
8. 设置RGB显示的颜色



作业1：

控制Robduino拓展板上的RGB灯显示7种常见颜色的**呼吸效果**

```
01.RGB_Control
#include <Adafruit_NeoPixel.h>  // 包含Adafruit NeoPixel库 Include Adafruit NeoPixel library

// 定义RGB控制引脚和数量 Define RGB control pins and quantity
#define RGB_PIN 6
#define RGB_NUM 1

// 枚举常见颜色 Enumerate common colors
enum ColorType {
  BLACK,
  RED,
  GREEN,
  BLUE,
  YELLOW,
  MAGENTA,
  CYAN,
  WHITE,
};

// 创建Adafruit_NeoPixel类的实例 Create an instance of the Adafruit_NeoPixel class
Adafruit_NeoPixel RGB = Adafruit_NeoPixel(RGB_NUM, RGB_PIN, NEO_GRB + NEO_KHZ800);

void setRGBColor(ColorType color);  // 设置RGB显示 Set the color of the RGB display
void setup() {
  RGB.begin();  // 初始化RGB Initialize RGB
  RGB.show();   // 刷新RGB显示 Refresh RGB display
}

void loop() {
  // 切换RGB显示颜色 Switch the color of RGB display
  for (int iRgbState = 0; iRgbState < 8; iRgbState++) {
    setRGBColor(iRgbState);
    delay(1000);
  }
}
void setRGBColor(ColorType color) {
  switch (color) {
    case RED:
      RGB.setPixelColor(0, RGB.Color(255, 0, 0));
      RGB.show();
      break;
    case GREEN:
      RGB.setPixelColor(0, RGB.Color(0, 255, 0));
      RGB.show();
      break;
    case BLUE:
      RGB.setPixelColor(0, RGB.Color(0, 0, 255));
      RGB.show();
      break;
    case YELLOW:
      RGB.setPixelColor(0, RGB.Color(255, 255, 0));
      RGB.show();
      break;
    case MAGENTA:
      RGB.setPixelColor(0, RGB.Color(255, 0, 255));
      RGB.show();
      break;
    case CYAN:
      RGB.setPixelColor(0, RGB.Color(0, 255, 255));
      RGB.show();
      break;
    case WHITE:
      RGB.setPixelColor(0, RGB.Color(255, 255, 255));
      RGB.show();
      break;
    default:
      RGB.setPixelColor(0, RGB.Color(0, 0, 0));
      RGB.show();
      break;
  }
}
```

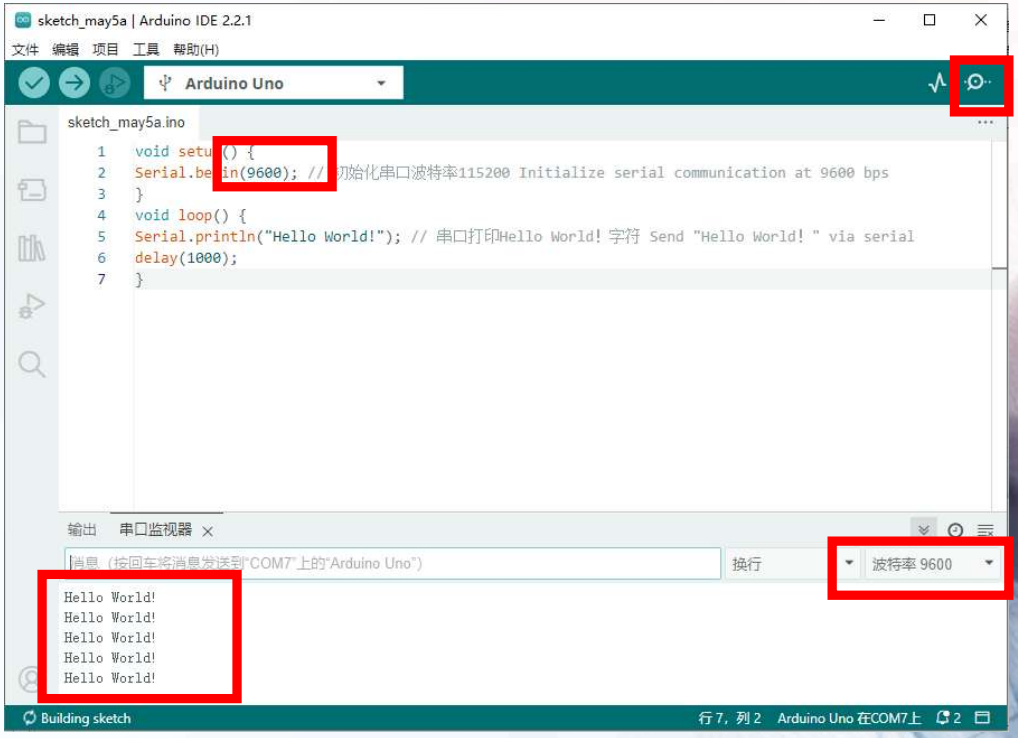
RGB控制

程序设计与编写

串口打印—Hello World!

设计目标：编写一段程序让Arduino 接受到我们发的指令就显示“Hello World！”字符串

```
void setup() {
Serial.begin(9600); // 初始化串口波特率115200
Initialize serial communication at 9600 bps
}
void loop(){
Serial.println("Hello World!"); // 串口打印
Hello World! 字符 Send "Hello World! " via
serial
delay(1000);
}
```



Tips：

波特率：串口通信每秒传输的**码元个数**，简单理解就是：**串口收发数据的传输速度**，收发双方**必须一模一样**，否则乱码。

常用的：9600 ---最通用、入门必用、最稳定；

115200 ---高速，项目常用；

其他：4800、19200、38400、57600

使用Serial.println()函数会自动换行；

使用Serial.print()函数不会自动换行。

程序设计与编写

按键控制蜂鸣器

设计目标：编写一段程序通过控制**Robduino**拓展板上的按键控制蜂鸣器。

1.播放音乐

控制原理：Arduino的**tone**库可以用于控制Arduino板上的数字引脚输出**指定频率的方波信号**，从而驱动连接到该引脚的蜂鸣器播放指定的音调。

```
#define BUZZER_PIN 10  // 定义蜂鸣器控制引脚 Define BUZZER control pins

void playMusic(int frequency, int duration);  // 播放具体频率和时长的音调 Play tones of a specific frequency and time

void setup() {
  pinMode(BUZZER_PIN, OUTPUT);  // 设置蜂鸣器引脚输出模式 Set the buzzer pin output mode
}
//音调频率 Tone frequency，持续时间 time of duration
void playMusic(int frequency, int duration) {
  tone(BUZZER_PIN, frequency);
  delay(duration);
  noTone(BUZZER_PIN); //让蜂鸣器停止响
}
```

```
void loop() {
  // Twinkle, twinkle, little star
  playMusic(523, 500);
  delay(100);
  playMusic(523, 500);
  delay(100);
  playMusic(587, 500);
  delay(100);
  playMusic(587, 500);
  delay(100);
  playMusic(659, 500);
  delay(100);
  playMusic(659, 500);
  delay(100);
  playMusic(587, 500);
  delay(1000);

  // How I wonder what you are
  playMusic(523, 500);
  delay(100);
  playMusic(493, 500);
  delay(100);
  playMusic(440, 500);
  delay(100);
  playMusic(440, 500);
  delay(100);
  playMusic(493, 500);
  delay(100);
  playMusic(493, 500);
  delay(1000);

  // Up above the world so high
  playMusic(523, 500);
  delay(100);
  playMusic(587, 500);
  delay(100);
  playMusic(659, 500);
  delay(100);
  playMusic(659, 500);
  delay(100);
  playMusic(587, 500);
  delay(100);
  playMusic(523, 500);
  delay(1000);
}
```

Tips：

蜂鸣器分为两种类型：有源蜂鸣器和无源蜂鸣器。

有源蜂鸣器：需要在外部提供工作电压，就能发出固定频率的声音；

无源蜂鸣器：需要外部电路提供驱动信号，通过改变信号的频率和幅度来控制声音的音高和响度。

程序设计与编写

按键控制蜂鸣器

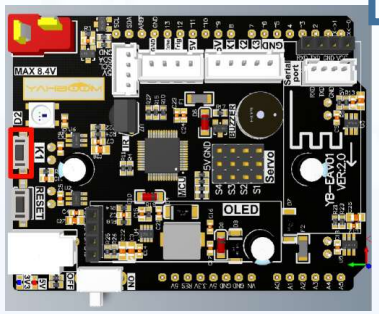
设计目标：编写一段程序通过控制Robduino拓展板上的按键控制蜂鸣器。

2.按键控制

控制原理：将按键状态和蜂鸣器关联在一起，按下并且松开按键就切换蜂鸣器状态。

```
// 定义按键引脚和控制状态 Define pin and
key(button) states
#define BUZZER_PIN 10
bool bBuzzerState = false;
// 定义按键引脚和控制状态 Define pin and
key(button) states
#define KEY_PIN 7
#define Press_KEY 0
#define Release_KEY 1
// 获取按键状态 Get key(button) status
int getKeyState(uint8_t pin);

void setup() {
  pinMode(KEY_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
}
```



```
void loop() {
  // 根据按键切换蜂鸣器发声 Switch the buzzer
sound according to the key
  if (getKeyState(KEY_PIN) == Press_KEY) {
    bBuzzerState = !bBuzzerState;
  }
  if (bBuzzerState) {
    tone(BUZZER_PIN, 1000);
    delay(20);
  }
  noTone(BUZZER_PIN);
}
```

// 按键按一次，翻转一次

// 固定频率的响声

作业2：

编写一段程序通过控制Robduino拓展板上的按键控制蜂鸣器播放一小段音乐《小星星》。

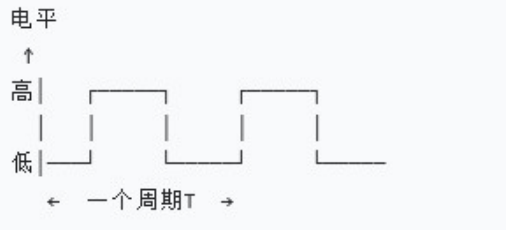
程序设计与编写

小车运动控制

1.电机控制

控制原理：通过改变控制电机信号引脚的PWM信号的占空比，从而控制电机旋转的角度。

PWM（脉冲宽度调制）——高低电平快速来回切换的方波



$$D = \frac{T_{on}}{T_{on} + T_{off}} \times 100\%$$

- T_{on}：高电平时间
- T_{off}：低电平时间
- $T = T_{on} + T_{off}$ 为一个周期



占空比D：一个 PWM 周期内，**高电平持续时间**占**整个周期**的百分比。占空比与等效平均电压 / 输出功率成正比。

PWM 占空比大小对电机的影响：

占空比越大 → 电机平均电压越高 → 转速越快、力气越大

占空比越小 → 电机平均电压越低 → 转速越慢、力气越小

PWM 频率：开关快慢（每秒跳变多少次）→ **不直接调速**，只影响运行质感

程序设计与编写

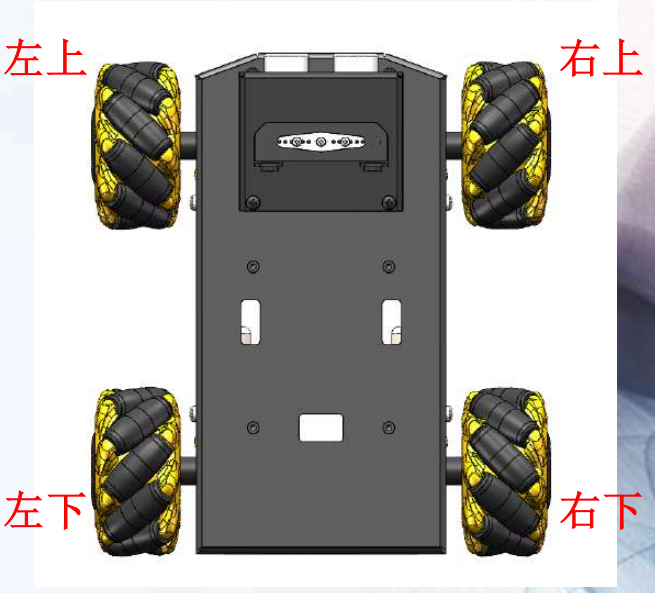
小车运动控制

1.电机控制

电机使用Robduino拓展板上的底层驱动芯片直接驱动，底层驱动芯片和Arduino Uno板采用I2C通信方式，I2C地址为0x40。

控制引脚

外设模块	I2C寄存器首地址编号（STC8GK2K64S4）	控制电机方向
L1（左上角电机）	11	控制正转（前进）
L1（左上角电机）	10	控制反转（后退）
L2（左下角电机）	8	控制正转（前进）
L2（左下角电机）	9	控制反转（后退）
R1（右上角电机）	13	控制正转（前进）
R1（右上角电机）	12	控制反转（后退）
R2（右下角电机）	14	控制正转（前进）
R2（右下角电机）	15	控制反转（后退）



程序设计与编写

小车运动控制

1.电机控制

电机使用Robduino拓展板上的底层驱动芯片直接驱动，底层驱动芯片和Arduino Uno板采用I2C通信方式，I2C地址为0x40。

- 包含Wire 和Adafruit_PWMServoDriver 库

```
#include <Wire.h>                    // 包含Wire(I2C)通讯库 Include Wire library
#include <Adafruit_PWMServoDriver.h> // 包含Adafruit PWMServoDriver库 Include
Adafruit PWMServoDriver library
```

- 定义电机控制引脚、I2C地址、电机频率以及初始速度

```
#define Motor_L1_F_PIN 11    // 控制小车左前方电机前进
。 。 。 。 。 。
#define Bottom_Layer_Driver_ADDR 0x40 // 定义底层芯片I2C设备地址
#define PWM_FREQUENCY 200    // 定义PWM频率
int MotorSpeed = 50;  // 定义电机速度
```

- 创建 Adafruit_PWMServoDriver类的实例

```
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(Bottom_Layer_Driver_ADDR);
```

```
void setMotorSpeed(uint16_t motor_forward_pin, uint16_t,
motor_backward_pin, int motor_speed);  // 设置单个电机速度
```

- 初始化

```
void setup() {
  Wire.begin();              // 初始化I2C通讯
delay(1000);
  pwm.begin();                        // PWM初始化
pwm.setPWMFreq(PWM_FREQUENCY);  // 设置PWM频率
}
```

程序设计与编写

小车运动控制

1.电机控制

电机使用Robduino拓展板上的底层驱动芯片直接驱动，底层驱动芯片和Arduino Uno板采用I2C通信方式，I2C地址为0x40。

- 循环函数

程序启动后，小车会按照下面规律一直重复：

1. 左边的第一个电机会正转500毫秒，然后停止；
2. 左边第二个电机会正转500毫秒，然后停止；
3. 右边的第一个电机会反转500毫秒，然后停止；
4. 右边的第二个电机会反转500毫秒，然后停止；
5. 所有电机停止5秒。

```
void loop() {
  // 控制小车左前方电机前进500毫秒 Control car left front motor forward 500
milliseconds
  setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, MotorSpeed);
  delay(500);
  setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);

  // 控制小车左后方电机前进500毫秒 Control car left rear motor forward 500
milliseconds
  setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, MotorSpeed);
  delay(500);
  setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);

  // 控制小车右前方电机后退500毫秒 Control car right front motor back 500
milliseconds
  setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -MotorSpeed);
  delay(500);
  setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);

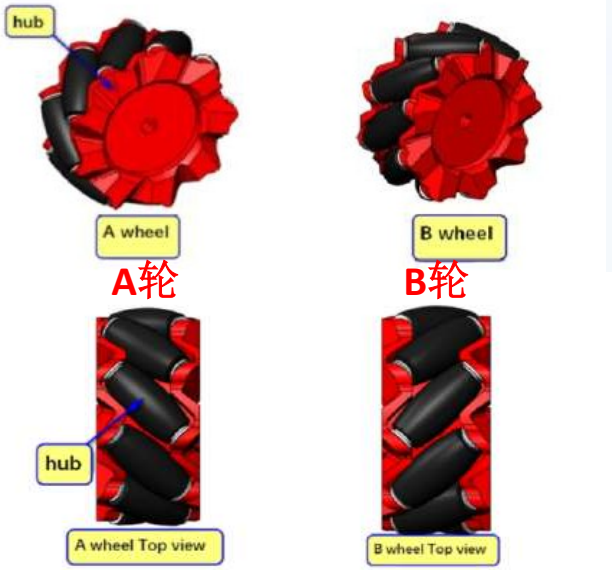
  // 控制小车右后方电机后退500毫秒 Control car right rear motor back 500 milliseconds
  setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -MotorSpeed);
  delay(500);
  setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);

  delay(5000);
}
```

程序设计与编写

小车运动控制

2.麦克纳姆轮简称麦轮，是由轮毂和围绕轮毂的辊子组成的，辊子是一种没有动力的从动小滚轮，麦克纳姆轮辊子轴线和轮毂轴线夹角是45度，并且有互为镜像关系的A、B轮两种，或者会被称为左旋轮和右旋轮。





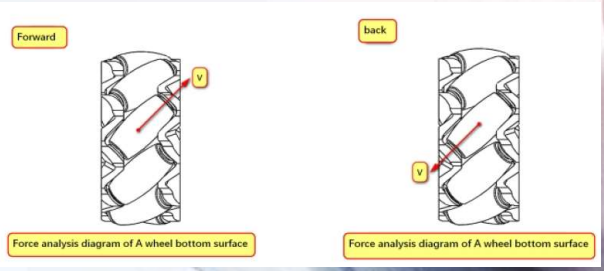
以小车车头为正方向，约定轮子前进时的方向为电机正转，轮子后退时的方向为电机反转。

实际运动方向需要根据麦轮和地面接触的部分判断

A轮：

前进：实际运动斜向右前方

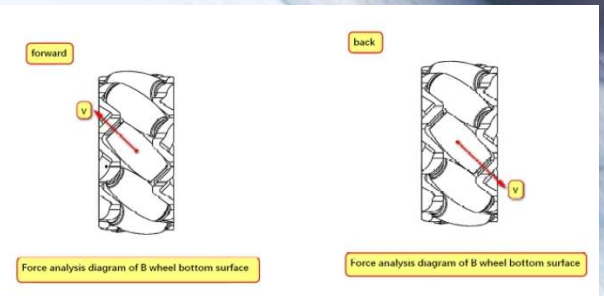
后退：实际运动斜向左后方



B轮：

前进：实际运动斜向左前方

后退：实际运动斜向右后方

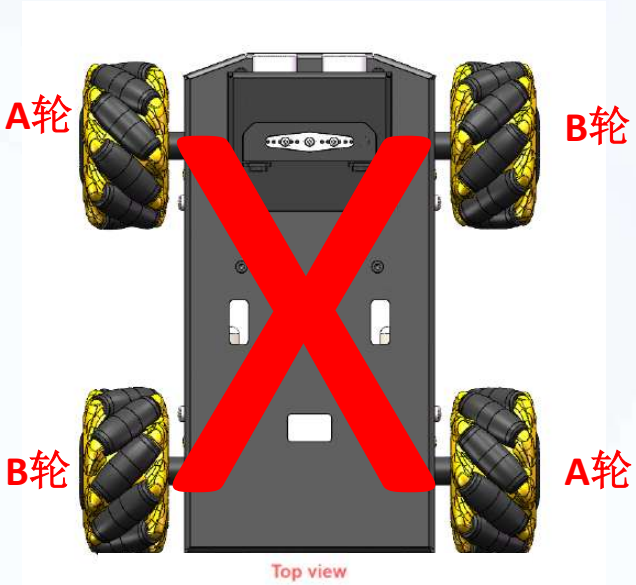


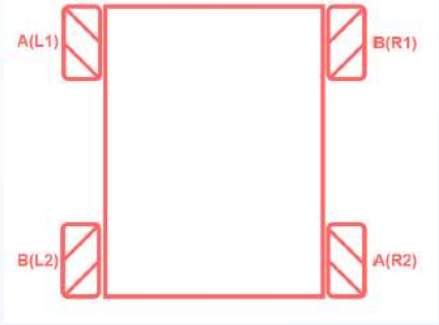
程序设计与编写

小车运动控制

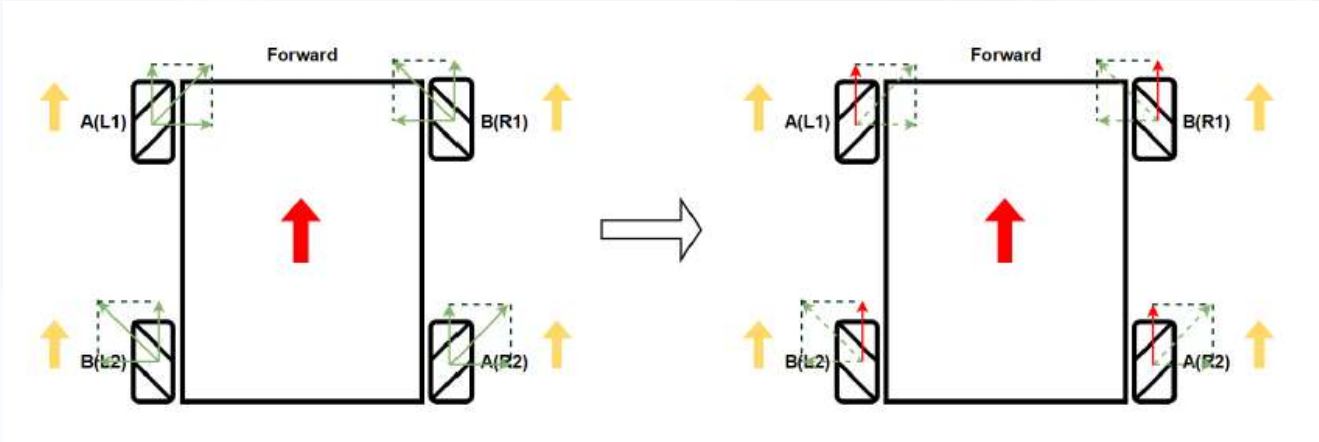
2.麦克纳姆轮小车运动状态分析

实际麦轮分布是**ABBA型**，实际安装可以参考图片中的俯视图进行安装，俯视图麦轮的棍子形状呈现“**X型**”。





小车前进的状态进行速度分解演示



黄色箭头：电机旋转方向(向上表示前进-电机正转、向下表示后退-电机反转)

红色箭头：小车实际运动方向

绿色箭头：速度分解方向(其中虚线箭头是抵消后的速度分量，红色的箭头是实际速度分量)

程序设计与编写

小车运动控制

2.麦克纳姆轮小车运动状态

实际麦轮分布是**ABBA型**，实际安装可以参考图片中的俯视图进行安装，俯视图麦轮的棍子形状呈现“**X型**”。

表格的运动状态是四个电机速度一致并且地面平坦的情况下分析的：

+代表正转、

-代表反转，

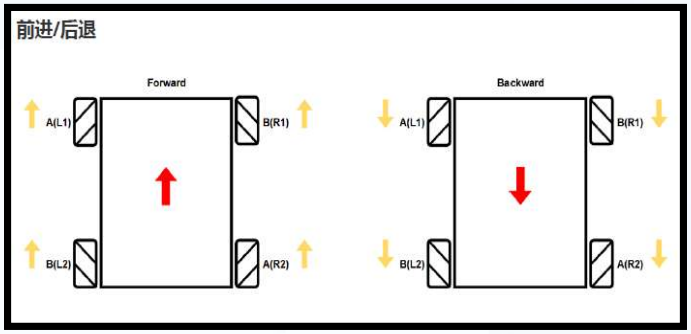
0代表无速度。

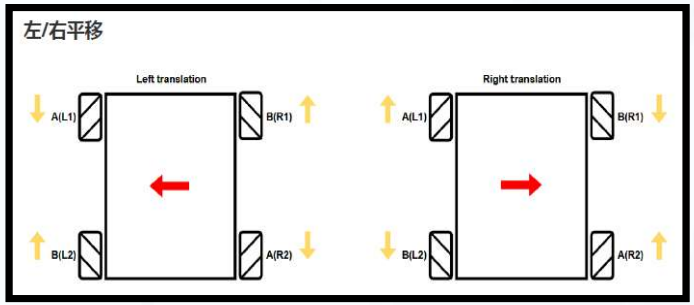
运动状态	L1	L2	R1	R2
前进	+	+	+	+
后退	-	-	-	-
左平移	-	+	+	-
右平移	+	-	-	+
左前方	0	+	+	0
左后方	-	0	0	-
右前方	+	0	0	+
右后方	0	-	-	0
逆时针自转（左旋）	-	-	+	+
顺时针自转（右旋）	+	+	-	-
停止	0	0	0	0

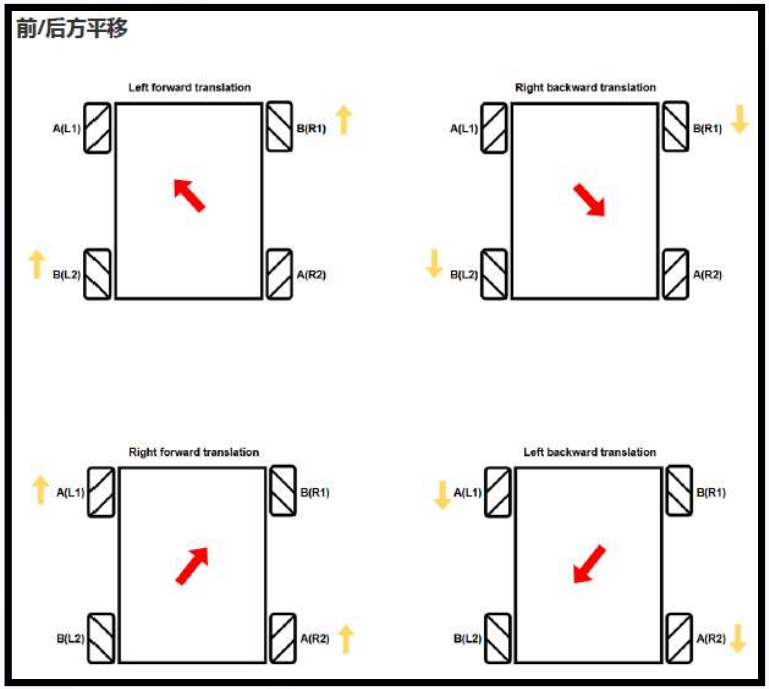
程序设计与编写

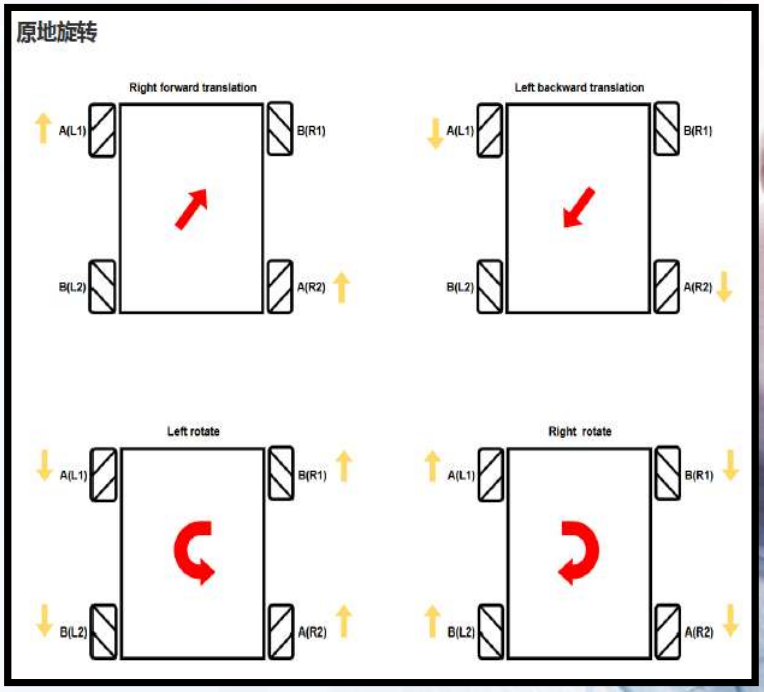
小车运动控制

2.麦克纳姆轮小车运动状态









◆小车实际运动过程中受结构、阻力以及摩擦力等因素影响；

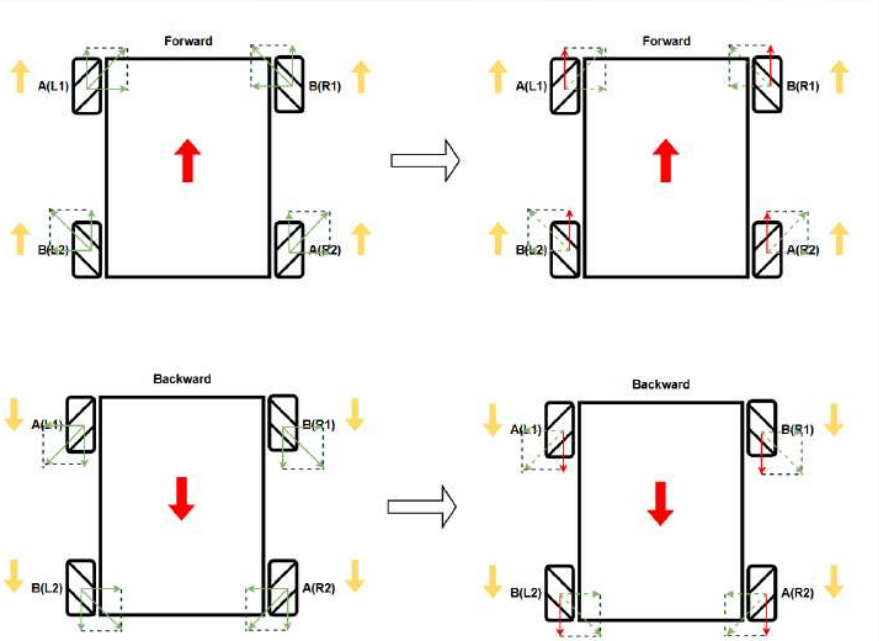
◆小车搭配的TTL电机无法进行PID调速；

部分运动状态并不是特别理想。

程序设计与编写

小车运动控制

3. 小车运动控制程序—前后运动为例



I. 包含Wire 、Adafruit_PWMServoDriver 库

II. 定义电机、按键控制引脚、I2C地址、电机频率以及初始速度

III. 枚举小车状态

```
enum OmniDirectionalCar {
  STOP,
  FORWARD,
  BACKWARD,
  LEFT,
  RIGHT,
  LEFT_ROTATE,
  RIGHT_ROTATE,
  LEFT_FORWARD,
  RIGHT_BACKWARD,
  RIGHT_FORWARD,
  LEFT_BACKWARD,
};
```

IV. 创建Adafruit_PWMServoDriver 的实例

I. 定义变量

```
void setMotorSpeed(uint16_t motor_forward_pin, uint16_t motor_backward_pin, int motor_speed);
 // 设置单个电机速度
void setCarMove(uint8_t Movement, int Speed); // 设置小车运动方式和速度
int getKeyState(uint8_t pin);  // 获取按键状态
bool setCarSwitch(); // 设置小车功能开关
```

程序设计与编写

小车运动控制

3. 小车运动控制程序—前后运动为例

- 初始化

```
void setup() {
  Wire.begin();                    // 初始化I2C通讯
  delay(1000);                     // 如果小车功能异常，可以增加这个
  pwm.begin();                     // PWM初始化
  pwm.setPWMFreq(PWM_FREQUENCY);   // 设置PWM频率
  setCarMove(STOP, 0);             // 设置小车停止状态
}
```

- 设置电机速度

```
void setMotorSpeed(uint16_t motor_forward_pin, uint16_t motor_backward_pin, int motor_speed) {
  motor_speed = map(motor_speed, -255, 255, -4095, 4095);
  if (motor_speed >= 0) {
    pwm.setPWM(motor_forward_pin, 0, motor_speed);
    pwm.setPWM(motor_backward_pin, 0, 0);
  } else if (motor_speed < 0) {
    pwm.setPWM(motor_forward_pin, 0, 0);
    pwm.setPWM(motor_backward_pin, 0, -(motor_speed));
  }
}
```

- 设置小车状态—设置小车运动方式和速度

```
void setCarMove(uint8_t Movement, int Speed) {
  switch (Movement) {
    case STOP:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Mot
      break;
    case FORWARD:
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      break;
    case BACKWARD:
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      break;
    case LEFT:
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      setMotorSpeed(Mot
      break;
    case RIGHT:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
      break;
    case LEFT_ROTATE:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
      setMotorSpee
      break;
    case RIGHT_ROT
      setMotorSpee
      setMotorSpee
      setMotorSpee
      setMotorSpee
      break;
    case LEFT_FORW
      setMotorSpee
      setMotorSpee
      etMotorSpee
      etMotorSpee
      reak;
    case RIGHT_BACKWARD:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
      break;
    case RIGHT_FORWARD:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
      break;
    case LEFT_BACKWARD:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
      break;
    default:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
      break;
  }
}
```

程序设计与编写

小车运动控制

3. 小车运动控制程序—前后运动为例

- 获取按键状态

```
int getKeyState(uint8_t pin) {
  if (digitalRead(pin) == LOW) {
    delay(20);
    if (digitalRead(pin) == LOW) {
      while (digitalRead(pin) == LOW)
        ;
      return Press_KEY;
    }
    return Release_KEY;
  } else {
    return Release_KEY;
  }
}
```

- 设置小车功能开关

```
bool setCarSwitch() {
  if (getKeyState(KEY_PIN) == Press_KEY) {
    Car_Switch = !Car_Switch;
  }
  return Car_Switch;
}
```

- 主函数---循环函数

```
void loop() {
  // 按键控制小车前后运动启动
  if (setCarSwitch()) {
    setCarMove(FORWARD, MotorSpeed);
    delay(1000);
    setCarMove(STOP, 0);
    delay(500);
    setCarMove(BACKWARD, MotorSpeed);
    delay(1000);
    setCarMove(STOP, 0);
    delay(500);
  } else {
    setCarMove(STOP, 0);
  }
}
```

程序设计与编写

小车循迹运动

设计目标：通过三路巡线模块获得小车相对于黑色路线的位置，并根据这个位置进行控制小车运动。

三路巡线模块：

集成了3路红外循线传感器的小车巡线模块。

红外光照射地面

白色地面：反光强 → 传感器输出低电平

黑色线条：吸光不反光 → 传感器输出高电平





三路巡线模块

三路巡线模块：

获取小车车头相对于黑色路线位置；

小车控制：

三路巡线模块位于黑色路线左边，小车左旋；
三路巡线模块位于黑色路线中间，小车直行；
三路巡线模块位于黑色路线右边，小车右旋。

➢ 小车巡线功能

```
/**
 * @brief 小车巡线功能 Car line patrol function
 * @param iLeftPin: 左边循迹传感器 Left tracking sensor
 * @param iMidPin: 中间循迹传感器 Middle tracking sensor
 * @param iRightPin: 右边循迹传感器 Right tracking sensor
 * @retval 无 None
 */
void PatrolCar(int iLeftPin, int iMidPin, int iRightPin) {
  int leftValue = analogRead(iLeftPin);
  int middleValue = analogRead(iMidPin);
  int rightValue = analogRead(iRightPin);

  // 进行简单的方向判断打印 The following is a simple judgment
  if ((leftValue < Threshold && middleValue > Threshold && rightValue <
Threshold) | (leftValue > Threshold && middleValue > Threshold && rightValue >
Threshold)) {
    setCarMove(FORWARD, iCarSpeed);
  } else if (leftValue > Threshold && middleValue < Threshold && rightValue <
Threshold) {
    setCarMove(LEFT_ROTATE, iCarSpeed);
  } else if (leftValue < Threshold && middleValue < Threshold && rightValue >
Threshold) {
    setCarMove(RIGHT_ROTATE, iCarSpeed);
  } else if (leftValue < Threshold && middleValue < Threshold && rightValue <
Threshold) {
    uTimeOut++;
    delay(20);
    if (uTimeOut > 100) {
      uTimeOut = 0;
      setCarMove(STOP, 0);
    }
  }
}
```

```
#define L_TRACK_PIN A2
#define M_TRACK_PIN A1
#define R_TRACK_PIN A0
```

Tips：

若小车巡线过程中抖动比较严重，可以尝试修改小车速度和三路巡线模块判断阈值。

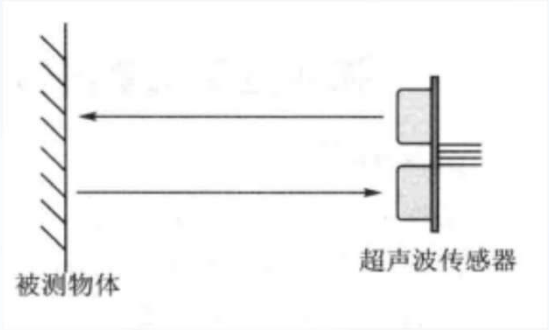
修改速度：直接在小车代码进行电机速度修改；

修改阈值：将三路巡线模块放在黑色背景和白色背景下，分别记录两种环境下的阈值，一般程序中的阈值取两者的中值，用户可以自行修改和调试。

程序设计与编写

超声波避障

设计目标：通过超声波模块获取前面物体的距离，并根据距离信息控制小车运动。




超声波传感器

超声波模块：：
获取小车前方物体距离；

小车控制：
小车前方物体小于设定的距离（**15cm**）：小车后退；
小车前方物体处于设定的距离区间（**15-30cm**）：小车停止；
小车前方物体大于设定的距离区间（**30cm**）：小车前进。

工作原理

1. 模块发出超声波
2. 碰到障碍物反射回来
3. 接收回波，输出**高电平**
4. 用高电平持续时间计算**距离**

$$距离(cm) = 时间(us) \times 1.7$$

空气中超声波速度：
常温 **25℃** 时：声速 ≈ **340 m/s**



➢ 获取距离：

```
/**
 * @brief 获取超声波距离 Get ultrasonic distance
 * @param trigPin: 触发测距引脚 Trigger pin
 * @param echoPin: 接收测距引脚 Echo pin
 * @retval 转换的距离 Measured distance (cm)
 */
float getDistance(int trigPin, int echoPin) {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH);
  float distance = (duration * 0.034 / 2);
  return distance;
}
```

程序设计与编写

超声波避障

设计目标：通过超声波模块获取前面物体的距离，并根据距离信息控制小车运动。

- 避障功能

```
/**
 * @brief 小车避障功能 Car obstacle avoidance function
 * @param 无 None
 * @retval 无 None
 */
void Car_Avoid() {
  Distance = getDistance(TRIG_PIN, ECHO_PIN);
  delay(10);
  if (Distance < NearDistance) {
    setCarMove(BACKWARD, iCarSpeed);
  } else if (NearDistance <= Distance && Distance <= FarDistance) {
    setCarMove(RIGHT_ROTATE, iCarSpeed);
  } else if (FarDistance < Distance) {
    setCarMove(FORWARD, iCarSpeed);
  } else {
    setCarMove(STOP, iCarSpeed);
  }
}
```

作业3（结课考查）：

编写程序使小车可以完成全向运动，并结合三路巡线模块和超声波避障模块等是小车在行进过程中可实现巡线和避障功能



THANK YOU
FOR ATTENTION